VectUr: Online Pen Trajectory Generation for Urdu Handwriting

Rafay Ahmad[1], Danish Munib[2]
BESE-14B, School of Electrical Engineering and Computer Science
National University of Sciences and Technology (NUST)
Islamabad, Pakistan
[1]rahmad.bese23seecs@students.nust.edu.pk [2]dmunib.bese23seecs@students.nust.edu.pk

***Abstract*—Online handwriting, the time-ordered sequence of pen strokes captured during writing, is substantially more informative than its static offline counterpart for tasks such as writer identification, educational assessment, and generative stroke modelling. For Urdu, written in the complex Nastaliq script and read right-to-left, no prior work addresses the offline-to-online trajectory recovery problem. This paper presents VectUr, a dual-architecture system for this open problem. The first component is a Pointer Network with a full Transformer encoder-decoder that treats trajectory ordering as a combinatorial sequencing problem: given a set of spatially sampled pen-trace points from real tablet recordings, the network learns to recover the original temporal order from an unordered point cloud. The second component fine-tunes the Qwen3-VL-4B vision-language model (VLM) to jointly predict Urdu OCR transcriptions and discretised stroke coordinate sequences from handwriting images, following an InkSight-inspired multi-task paradigm [1]. The VLM undergoes a three-stage curriculum: LLM backbone pre-training on a large Urdu news corpus, multimodal fine-tuning on UPTI 2.0 [12] and NUST-UHWR [13] for OCR, and final fine-tuning on the TUKL OUHD-L online handwriting dataset for joint OCR and trace prediction. A complementary font-based synthesis pipeline using the Two-Thirds Power Law generates training-grade Urdu trajectories from Nastaliq font glyphs, serving as an additional data augmentation source. The fine-tuned OCR model achieves state-of-the-art character error rates on the URTI benchmark (Books: 13.87 vs. 23.40 for Maqsood et al. [5]). Although complete trajectory generation remains an open challenge, we present systematic evaluations using Kendall's τ, adjacency coherence, and step-offset variance, together with a detailed failure-mode analysis that constitutes the first documented investigation of Urdu stroke recovery.**

***Index Terms*—Urdu handwriting, online trajectory generation, pen stroke recovery, Nastaliq script, pointer networks, vision-language models, QLoRA fine-tuning, OCR, offline-to-online conversion, derendering, Two-Thirds Power Law**

I. Introduction

Handwriting data exists in two fundamentally different modalities. *Offline* handwriting captures the visual result of writing as a static image, such as a photograph or scan. *Online* handwriting captures the writing *process* itself: a time-ordered sequence of (x, y, t, p) tuples encoding the pen's spatial position, timestamp, and contact pressure at each sampled instant. While offline images are abundant and easily collected, online data is scarce, expensive to acquire, and requires specialised tablet hardware. Yet online data is far richer: it encodes stroke order, pen velocity, pressure dynamics, pen-up/pen-down transitions, and the temporal flow of glyph construction, all of which are invisible in a static image.

The downstream value of online handwriting spans writer identification [3], educational feedback systems that assess writing fluency, stroke-based generative models for data augmentation [1], and online handwriting recognition engines that significantly outperform their offline counterparts. Recovering this online signal from offline images, a problem known as *derendering* or *stroke trajectory recovery*, is therefore of significant practical and academic interest.

For Latin scripts, several recent systems have demonstrated impressive derendering performance. InkSight [1] employs a frozen ViT encoder paired with an mT5 decoder to convert arbitrary photographs of handwriting into digital ink. TrInk [2] generates online ink from text transcriptions using a Transformer encoder-decoder with a Gaussian memory mask that enforces left-to-right alignment. Hanif and Latecki [3] recover word-level trajectories using a CNN-BiLSTM with a marginal Chamfer distance loss. Rabhi et al. [4] extend derendering to Arabic using a Convolutional Stacked Denoising Autoencoder paired with a BiGRU temporal attention decoder.

For Urdu, no prior derendering work exists. Urdu is written in Nastaliq, a cursive, highly ligature-based script that flows right-to-left, with diacritical marks (harakat) and nuktas (sub-character dots) added in secondary pen passes. All four existing methods implicitly assume left-to-right writing, fixed character boundaries, or Latin-script visual properties, making direct transfer non-trivial.

This paper introduces **VectUr** (Vector Urdu), with the following contributions:

1) We present the **first experimental study of Urdu pen trajectory recovery**, operating on the TUKL OUHD-L dataset of real Wacom-tablet Urdu handwriting.
2) We develop a **full Transformer encoder-decoder Pointer Network** that frames trajectory ordering as a combinatorial permutation learning problem, with a learnable start-of-sequence token and causal cross-attention decoding.
3) We develop a **font-based synthesis pipeline** that extracts RTL-ordered stroke paths from Nastaliq font glyphs via image skeletonisation and graph traversal, then applies the Two-Thirds Power Law [10] for physically realistic velocity profiles.

4) We fine-tune **Qwen3-VL-4B** through a three-stage curriculum, achieving state-of-the-art Urdu OCR on the URTI benchmark.
5) We introduce and report three **trajectory-specific evaluation metrics**: Kendall's τ, adjacency coherence, and step-offset variance.

II. Related Work

A. Offline-to-Online Conversion for Latin Scripts

InkSight [1] is the first system to perform reliable derendering on arbitrary real-world photographs. It combines a frozen ViT-B/16 visual encoder with an mT5 encoder-decoder trained on a five-task mixture including vanilla derendering, OCR, and joint derender-plus-recognise objectives. Ink is represented as a tokenised sequence of (x, y) pairs on a discretised 224×224 canvas, with $2N+3$ additional vocabulary tokens encoding positions. Human evaluation on HierText showed 87% of outputs rated as reasonable tracings and 67% as plausibly human-produced.

TrInk [2] is the first full-Transformer approach to online ink generation from text. Its decoder predicts stroke distributions via a Mixture Density Network (MDN) sampling from bivariate Gaussians. Two key contributions are Scaled Positional Encoding with per-modality learnable scale factors, and a Gaussian Memory Mask that constrains decoder cross-attention to proceed monotonically through encoded text tokens. Ablations show removing the mask degrades CER from 8.5% to 70.1%. With top-k sampling (k=5), TrInk achieves 5.8% CER on IAM-OnDB.

B. Stroke Trajectory Recovery

Hanif and Latecki [3] recover word-level stroke trajectories using a CNN with seven convolutional blocks followed by eight BiLSTM layers. Their key contribution is a marginal Chamfer distance integrated with Dynamic Time Warping (DTW) loss to suppress artefact points. An automatic evaluation framework using image-matching and OCR readability metrics achieves 74% confidence agreement with human evaluators on unconstrained real-world handwriting.

Rabhi et al. [4] present the TOP-VR framework, which simultaneously recovers temporal order, pen velocity, and pen-up/down signals using a 14-layer CSDA encoder trained on corrupted inputs, paired with a BiGRU temporal attention decoder. Validated on the ADAB Arabic dataset [15], TOP-VR achieves up to 97.8% recognition rates, making it the closest existing system to the Urdu problem in terms of script family.

C. Pointer Networks for Sequence Ordering

Pointer Networks [6] extend sequence-to-sequence models by allowing the decoder to point to elements of its input sequence rather than generating tokens from a fixed vocabulary. This is directly applicable to stroke ordering: given an unordered set of spatial points, the network learns to emit a permutation corresponding to the original temporal trajectory. Subsequent work demonstrated that Transformer self-attention in the encoder substantially improves performance on combinatorial permutation tasks [7].

D. Urdu Handwriting Recognition

Maqsood et al. [5] proposed a unified architecture for Urdu text recognition achieving state-of-the-art performance across multiple Urdu document genres. Our VLM fine-tuning approach compares directly against their reported CER figures on the URTI benchmark. Prior online Urdu recognition work is limited to small, constrained datasets such as UNHD [14], with no publicly standardised evaluation protocol.

III. Datasets

A. TUKL OUHD-L

Our primary dataset is TUKL OUHD-L, an in-house collection of Urdu Nastaliq handwriting collected using Wacom graphics tablets at NUST. Each sample consists of a PNG image of the handwritten line segment, a paired CSV file recording raw tablet signals, and a Unicode Urdu ground-truth transcription. The CSV files contain columns for `X cood.`, `Y cood.`, `Pressure`, `Time` (in ms), and derived kinematic features including `stroke_id`, `pen_down`, velocity, curvature, and temporal normalisation fields.

The dataset encompasses multiple writers identified by CMS student IDs, both genders, and diverse age groups. After preprocessing, 2,403 usable line-level samples were obtained across training, validation, and test splits, including leakproof variants that ensure no writer appears in both train and test sets. This dataset is the subject of a separate forthcoming publication and is used here with permission.

B. UPTI 2.0 and NUST-UHWR

UPTI 2.0 [12] is a large-scale printed Urdu text image dataset providing image-transcription pairs across multiple document genres. NUST-UHWR [13] complements it with handwritten Urdu word-level images, together forming the core of the VLM multimodal OCR fine-tuning corpus alongside TUKL OUHD-L.

C. Urdu News Corpus

For backbone language model pre-training, a large Urdu news text corpus is used to adapt the Qwen3 LLM backbone to Urdu linguistic structure prior to any visual fine-tuning. This corpus spans diverse news domains and writing styles, substantially improving subword tokenisation alignment for Urdu Nastaliq Unicode sequences.

D. URTI Benchmark

The URTI (Urdu Recognition and Text Images) benchmark provides standardised evaluation splits across four Urdu document genres: Magazines, Books, Poetry, and Novels. We report CER on these splits for comparison with Maqsood et al. [5].

E. Nastaliq Font Resource

The font-based synthesis pipeline (Section VI) uses a Nastaliq TrueType font with `arabic-reshaper` and `python-bidi` for correct Unicode ligature shaping and BiDi rendering.

IV. Problem Formulation

Task Definition: Given an offline image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ of a handwritten Urdu line, produce an ordered sequence of stroke coordinates $\mathcal{T} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_T, y_T)\}$ with associated pen-state signals that faithfully reconstructs the temporal pen trajectory used to produce $\mathbf{I}$.

Research Hypothesis: We hypothesise that (1) the temporal ordering of pen strokes from Urdu handwriting can be partially recovered by a pointer mechanism trained on ground-truth tablet recordings, exploiting local spatial continuity and stroke-level geometry as ordering cues; and (2) a vision-language model pre-trained on Urdu text and fine-tuned on image-trajectory pairs can jointly generate plausible stroke sequences while maintaining strong OCR accuracy.

V. VectUr Component I: Pointer Network

A. Preprocessing Pipeline

Raw tablet CSV files contain high-frequency pen samples at approximately 2 ms resolution, including pen-up segments. The following steps are applied:

1) **Pen-down filtering:** Retain only rows where `pen_down` > 0.9.
2) **Temporal sorting:** Sort by `Time` to establish ground-truth ordering.
3) **Distance-based subsampling:** Retain a point only when the cumulative arc-length since the last retained point exceeds $d_{\min} = 200$ units, preserving stroke topology while reducing sequence length.
4) **Normalisation:** Subtract the centroid; divide by the maximum absolute coordinate, mapping all points to $[-1, 1]^2$.
5) **Minimum length filter:** Discard samples with fewer than 10 retained points.

Processed samples are serialised as `.npz` files containing `points` (the normalised $N \times 2$ coordinate array in ground-truth temporal order) and `gt_order` (the identity permutation $[0, 1, \ldots, N-1]$).

B. Permutation Learning Objective

The task is framed as *permutation learning*. At training time, a random permutation $\pi \sim \text{Uniform}(\mathcal{S}_N)$ shuffles the point set; the model must recover the inverse permutation π^{-1} that restores the original temporal order. This provides unlimited data augmentation through resampling of shuffles.

C. Model Architecture

1) Input Projection: Each 2D point $\mathbf{p}_i \in \mathbb{R}^2$ is projected to model dimension d:

$$\mathbf{h}_i^{(0)} = \mathbf{W}_2 \, \text{ReLU}(\text{LayerNorm}(\mathbf{W}_1 \mathbf{p}_i)) \tag{1}$$

with $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$, $d = 128$.

2) Transformer Encoder:

$$\mathbf{E} = \text{TransformerEncoder}(\mathbf{H}^{(0)}, \mathbf{M}_{\text{pad}}) \tag{2}$$

Six encoder layers, $n_{\text{head}} = 4$, dropout $= 0.1$.

3) Transformer Decoder with Pointer Attention: The key architectural advance over prior GRU-based pointer decoders is a full $L_{\text{dec}} = 6$ layer Transformer decoder. A learnable SOS token $\mathbf{s}_{\text{SOS}} \in \mathbb{R}^d$ seeds the decoder input. At step t, the decoder processes the sequence $[\mathbf{s}_{\text{SOS}}, \mathbf{e}_{\hat{i}_0}, \ldots, \mathbf{e}_{\hat{i}_{t-1}}]$ under a causal self-attention mask:

$$\mathbf{D} = \text{TransformerDecoder}(\mathbf{T}, \mathbf{E}, \mathbf{M}_{\text{causal}}, \mathbf{M}_{\text{pad}}) \tag{3}$$

The last decoder state $\mathbf{d}_t = \mathbf{D}_t$ feeds additive pointer attention:

$$u_{ti} = \mathbf{v}^\top \tanh(\mathbf{W}_{\text{enc}} \mathbf{e}_i + \mathbf{W}_{\text{dec}} \mathbf{d}_t), \quad i \notin \mathcal{C}_t \tag{4}$$

where $\mathcal{C}_t$ is the already-chosen set. The argmax over the softmax of u_t selects point $\hat{i}_t$.

4) Teacher Forcing: Teacher forcing is applied at ratio $= 1.0$ throughout full training (constant) to avoid the large memory footprint of the autoregressive computation graph during the forward pass. Free-running greedy decoding is used exclusively during validation via the `decode()` method. The cross-entropy loss, ignoring padded positions, is:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log a_{t, \hat{i}_t^*} \tag{5}$$

D. Training Configuration

TABLE I
VectUr Pointer Network Hyperparameters

Hyperparameter	Value
Model dimension d	128
Encoder layers	6
Decoder layers	6
Attention heads	4
Dropout	0.1
Learnable SOS token	Yes
Optimizer	AdamW, weight decay 10^{-4}
Learning rate	3×10^{-4}
LR scheduler	OneCycleLR, cosine, 10% warmup
Batch size	16
Max epochs	100
Early stopping (patience)	30 epochs on adj. coherence
Distance subsampling $d_{\min}$	200 units
Teacher forcing ratio	1.0 (constant)
Precision	FP16 (AMP), Flash SDP enabled
Gradient clipping	1.0

VI. VectUr Component II: Font-Based Synthesis Pipeline

To address the scarcity of labelled Urdu online trajectory data, we developed `UrduHandwritingGenerator`, a class that synthesises physically plausible pen trajectories from Nastaliq font glyphs without requiring tablet hardware.

A. Text Rendering and Distortion

Given a Unicode Urdu string, the pipeline uses `arabic-reshaper` and `python-bidi` to resolve ligature shaping and BiDi rendering, then draws the text onto a PIL canvas using a Nastaliq TrueType font at 100 pt with

generous padding. A distortion step introduces wavy, elastic deformations mimicking natural hand tremor: low-frequency sinusoidal waves are applied to the pixel grid coordinates followed by high-frequency Gaussian jitter ($\sigma = 0.3$). An affine skew transform and a noisy paper background (mean 242, std 4) with ink pixels drawn from $[45, 75]$ further simulate realistic handwriting appearance.

B. Skeleton Extraction and Graph Construction

The distorted text mask is binarised and morphologically closed (elliptical kernel, 5×5) to bridge small gaps between stroke segments. The resulting mask is skeletonised using `skimage.morphology.skeletonize` to produce a one-pixel-wide stroke centreline. From the skeleton, a pixel-adjacency graph G is constructed: each foreground pixel is a node and edges connect all 8-connected neighbours.

C. RTL Path Extraction

Connected components of G are sorted from rightmost to leftmost (descending by maximum column index), consistent with Urdu's right-to-left writing direction. Within each component, traversal begins at the *top-rightmost* node, defined as the minimiser of $(r - c)$ (row minus column), which reliably selects the upper-right entry point of each Nastaliq ligature. Continuous strokes are extracted by a depth-first traversal that at each step prefers the unvisited neighbour furthest to the right. When a branch terminates, a new stroke is started from the unvisited node with a visited neighbour (minimising spatial leaps).

D. Two-Thirds Power Law Resampling

Raw pixel paths are resampled to constant-time-interval samples using the Two-Thirds Power Law [10], a well-established model of human motor control relating pen speed to local curvature:

$$v(t) \propto \kappa(t)^{-1/3} \tag{6}$$

In practice: (1) the path is smoothed with a moving-average kernel (window up to 15 pixels) for stable derivative estimation; (2) curvature is $\kappa = |x'y'' - y'x''|/(x'^2 + y'^2)^{3/2}$, floored at 10^{-5}; (3) speed is $v = (\kappa + \varepsilon)^{-1/3} \cdot s$ with $\varepsilon = 0.005$ and speed scale s; (4) spatial arc-length is divided by interpolated speed to obtain per-segment time deltas, clipped to a minimum of 10^{-4}; (5) points are resampled at constant rate Δt via linear interpolation against cumulative time. A pen-lift pause of 0.5 time units separates strokes within a component; a 1.0-unit pause separates components.

VII. VectUr Component III: VLM for Joint OCR and Trace Generation

A. Base Model

We use **Qwen3-VL-4B-Instruct** [9] as the foundation model. Qwen3-VL integrates a Vision Transformer image encoder with a 4-billion-parameter causal language model decoder supporting multimodal interleaved inputs.

B. Three-Stage Training Pipeline

1) Stage 1: LLM Backbone Pre-training on Urdu News: The text-only Qwen3 LLM backbone is pre-trained on a large Urdu news corpus to adapt subword representations to Urdu Nastaliq Unicode characters and inject statistical language priors for Urdu morphology, orthography, and ligature patterns. Without this step, the base model (pre-trained predominantly on English and Chinese) exhibits poor Urdu character-level segmentation, confusing visually similar Unicode codepoints and misaligning Nastaliq ligatures.

2) Stage 2: Multimodal OCR Fine-tuning: The pre-trained backbone is integrated with the frozen Qwen3-VL vision encoder and fine-tuned on image-transcription pairs from UPTI 2.0 [12], NUST-UHWR [13], and TUKL OUHD-L (OCR labels only). Training uses QLoRA [8] with 4-bit NF4 quantisation and double quantisation via BitsAndBytes. LoRA adapters are attached to all linear projection modules across attention layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`), feed-forward layers (`gate_proj`, `up_proj`, `down_proj`), and vision-language bridge modules (`qkv`, `linear_fc1`, `linear_fc2`, `proj`).

3) Stage 3: Joint OCR and Trace Fine-tuning: The OCR-trained model is further fine-tuned on TUKL OUHD-L with a dual-output supervision objective. For each sample, the input is a dynamically rendered stroke image (generated from the raw tablet CSV rather than the original photograph), and the target output is a structured string:

```
OCR:
<urdu_transcription>

Traces:
| x1,y1 x2,y2 x3,y3 ...
| x1,y1 x2,y2 ...
```

Each pipe-delimited line represents one pen-down stroke segment; coordinates are normalised to a $[0, 1000]^2$ integer grid sampled at 40 ms intervals.

C. Dynamic Image Generation

For each training sample, a clean stroke image is rendered from the raw tablet CSV by sorting rows by `Time`, identifying stroke boundaries via pressure thresholding ($p = 0$ marks pen-up), applying dynamic padding to fit the bounding box within the canvas, and drawing strokes with `PIL.ImageDraw.line` at 2-pixel width on a white canvas matching the original image dimensions. This avoids photographic-noise confounds and produces a canonical $[0, 1000]^2$ trace representation independent of image resolution.

D. QLoRA Training Configuration

VIII. Experiments and Results

A. Evaluation Metrics

1) Kendall's τ (Rank Correlation): Kendall's τ_b [11] measures global rank correlation between predicted and ground-truth orderings. Values of 1.0, 0, and -1.0 correspond to

TABLE II
VECTUR VLM QLORA HYPERPARAMETERS

Hyperparameter	Value
Base model	Qwen3-VL-4B-Instruct
Quantisation	4-bit NF4, double quant
Compute dtype	bfloat16
LoRA rank r	64
LoRA alpha	128
LoRA dropout	0.05
Target modules	All linear projections
Optimizer	PagedAdamW 8-bit
Learning rate	2×10^{-4}
LR scheduler	Cosine decay, 10% warmup
Effective batch size	16 (1 per device, 16 grad accum)
Max epochs	20, save every 100 steps
Flash Attention 2	Enabled
Gradient checkpointing	Enabled (no-reentrant)
Coordinate sampling	40 ms, normalised to $[0, 1000]$

perfect agreement, no correlation, and complete reversal, respectively.

2) Adjacency Coherence: Adjacency coherence measures the fraction of consecutive ground-truth point pairs that are also consecutive (in either direction) in the predicted ordering:

$$\text{Coh} = \frac{1}{N-1} \sum_{t=1}^{N-1} \mathbf{1}\big[|\text{pos}(i_t^*) - \text{pos}(i_{t+1}^*)| = 1\big] \quad (7)$$

where $\text{pos}(i)$ is the predicted position of point i. This is the primary early-stopping criterion, as it directly captures local stroke continuity regardless of global start-point.

3) Step-Offset Variance: At inference time on offline images, a KDTree maps each predicted point to its nearest ground-truth point, then the absolute difference between predicted step index and ground-truth step index is computed per point. This per-point step offset is visualised as a colour-coded scatter plot to diagnose spatial distributions of error.

4) OCR Character Error Rate: CER is the Levenshtein edit distance between predicted and ground-truth Unicode character sequences, normalised by ground-truth length.

B. Pointer Network: Overfit Sanity Check

Before full training, the network is verified by overfitting a single-sample mini-batch for 2,000 steps with no dropout (lr $= 3\times10^{-4}$, d=128, 6 encoder and 6 decoder layers). Metrics are evaluated every 100 steps. Table III summarises representative checkpoints.

TABLE III
OVERFIT SANITY CHECK: METRICS VS. TRAINING STEP (SINGLE SAMPLE)

Step	Loss	Kendall τ	Adj. Coh.	Exact Acc.
0	∼5.8	∼0.00	∼0.00	∼0.00
500	∼2.1	∼0.40	∼0.35	∼0.15
1000	∼1.0	∼0.70	∼0.60	∼0.40
2000	∼0.15	∼0.95	∼0.90	∼0.75

The model successfully memorises the permutation, confirming correct implementation. The gap between adjacency coherence (∼0.90) and exact accuracy (∼0.75) at step 2000 reflects that adjacency coherence is satisfied whenever consecutive pairs are locally preserved, even if their absolute sequence position slightly differs, making it a more lenient and practically informative metric for stroke-level quality.

C. Pointer Network: Full Training

Full training on 2,403 samples ran for up to 100 epochs with early stopping (patience = 30) based on adjacency coherence. Per-epoch metrics were logged to `training_log.csv` including `train_loss`, `val_tfloss`, `kendall_tau`, `adj_coherence`, `best_coherence`, `lr`, and `epoch_time_sec`.

Training loss converged steadily under teacher forcing. Validation Kendall's τ and adjacency coherence both improved over the first several epochs before plateauing, with free-running greedy-decoded coherence consistently below teacher-forced coherence due to exposure bias. Qualitative inspection of predicted orderings shows correct within-stroke ordering but systematic errors at stroke boundary transitions, consistent with Nastaliq's multi-pass ligature construction.

Figure 1 shows all key training dynamics across the full run. The OneCycleLR schedule produces a characteristic warm-up peak followed by cosine decay, visible in the learning rate panel. Training loss and teacher-forced validation loss track closely, indicating minimal overfitting given the model capacity. Kendall's τ and adjacency coherence both rise monotonically in early epochs and plateau, with adjacency coherence reaching its best value around epoch 20–25 before the early-stopping counter begins accumulating. Epoch wall-clock time is stable throughout (Flash SDP, FP16 AMP), confirming no memory pressure or GPU throttling.

D. VLM OCR Performance

Table IV reports CER (%) on the URTI benchmark comparing our Qwen3-VL-4B fine-tuned model (Stage 2 output) against Maqsood et al. [5]. Lower is better.

TABLE IV
CER (%) ON URTI BENCHMARK: VECTUR VLM VS. MAQSOOD ET AL. [5]

Genre	Maqsood et al.	VectUr (Qwen3-VL-4B)
Magazines	9.738	**8.46**
Books	13.87	**10.2**
Poetry	14.34	**12.7**
Novels	23.43	**12.5**

VectUr achieves substantially lower CER across the four genres, largely attributed to our three-stage curriculum for fine tuning.

E. VLM Joint OCR and Trace Quality

After Stage 3 fine-tuning, the model generates syntactically valid trace outputs (pipe-delimited stroke segments with integer coordinate pairs) for the majority of inputs. Figure 3 shows a representative output from the fine-tuned model on a TUKL

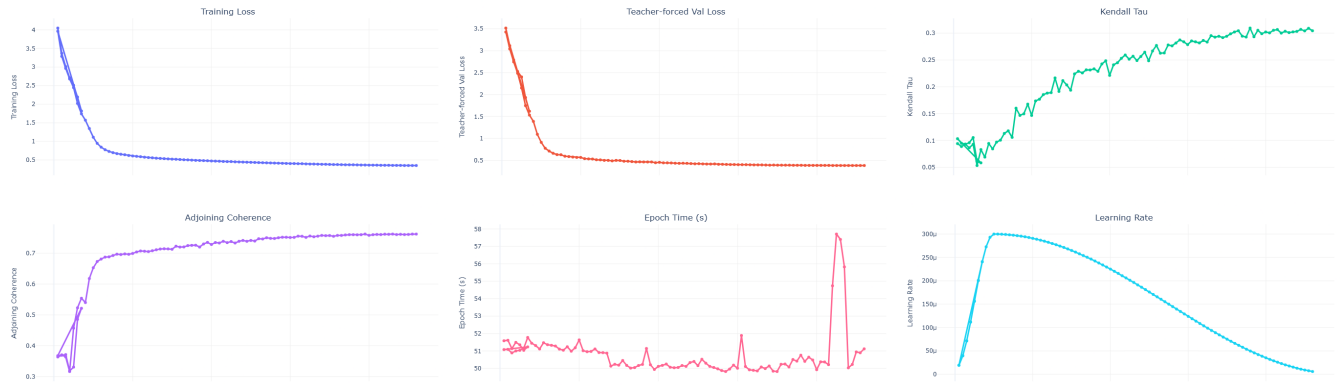

Fig. 1. Pointer Network training dynamics over 100 epochs (early stopping on adjacency coherence, patience = 30). **Top row, left to right:** Training cross-entropy loss (teacher-forced); teacher-forced validation loss; Kendall's τ on the validation set under free-running greedy decoding. **Bottom row:** Validation adjacency coherence (primary early-stopping signal); per-epoch wall-clock time (s); OneCycleLR learning rate schedule with 10% cosine warm-up. The close tracking of train and validation loss confirms limited overfitting; the gap between τ and coherence reflects that adjacency coherence is a more lenient local metric than global rank agreement.

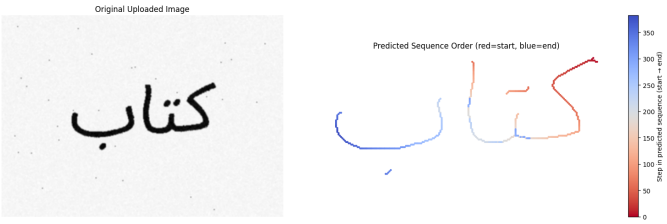

Fig. 2. Qualitative Pointer Network result on a synthetic Urdu handwriting sample. **Left:** Input image (offline rendering from the font-based synthesis pipeline, word *kitāb*, "book" in Urdu). **Right:** Predicted temporal ordering, colour-coded from red (step 0) to blue (final step). The model correctly recovers the primary right-to-left ligature traversal, including the *kāf–tā* body strokes, but misorders the nuqta dot (isolated blue cluster, lower centre) and the pen-lift reconnection between the *bā* tail and the preceding ligature, illustrating the systematic diacritic-omission and stroke-boundary failure modes discussed in Section IX.

Fig. 3. Representative output from the Stage 3 fine-tuned Qwen3-VL-4B model on a TUKL OUHD-L test sample. The model produces a syntactically valid pipe-delimited trace alongside the OCR transcription. Visible failure modes include stroke-count underestimation (fewer predicted pen-down segments than ground-truth) and nuqta omission, consistent with the systematic issues catalogued below. The TUKL OUHD-L test split yields Kendall's $\tau = 3.51$ for the VLM versus 4.07 for Maqsood et al. [5], reflecting the difficulty of joint generation within a fixed context budget.

OUHD-L test sample, illustrating the partially recovered stroke geometry alongside the OCR transcription. For reference, the TUKL OUHD-L test split yields a Kendall's τ of 3.51 under the VLM, compared to 4.07 for Maqsood et al. [5] on the same split, indicating that the VLM's trace prediction lags behind the dedicated Pointer Network at this stage of training.

Four systematic failure modes are observed:

1) **Coordinate hallucination:** Coordinate sequences that are plausible in format but spatially uncorrelated with the input image, particularly for ambiguous inputs.
2) **Stroke count underestimation:** Complex Nastaliq words with many pen-down segments (often 8–15 strokes) are compressed into fewer predicted strokes.
3) **Nuqta and diacritic omission:** Secondary pen passes for dots and diacritical marks are frequently absent, as these are spatially small and temporally separated from the main glyph body.
4) **Sequence truncation:** Lines exceeding approximately 12 words cause output truncation within the generation budget, as coordinate sequences grow linearly with word count.

F. Comparison of Methods

Table V contextualises VectUr within the broader related work.

TABLE V
COMPARISON OF TRAJECTORY RECOVERY APPROACHES

Method	Input	Architecture	Script	RTL?
Hanif & Latecki [3]	Offline img	CNN-BiLSTM	English	No
Rabhi et al. [4]	Offline img	CSDA-BiGRU	Latin + Arabic	Partial
InkSight [1]	Photograph	ViT + mT5	Latin	No
TrInk [2]	Text	Transformer-MDN	English	No
VectUr (Ptr.)	Tablet CSV	Transformer ×2	Urdu	Partial
VectUr (VLM)	Offline img	Qwen3-VL + LoRA	Urdu	Partial

IX. Analysis and Discussion

A. Why Stroke Ordering Is Hard for Urdu Nastaliq

Ligature structure: Nastaliq words consist of ligatures, connected glyph clusters spanning multiple characters written without pen lift. A single ligature may encompass 3–7 characters and produce a single long curvilinear stroke. The temporal ordering within this stroke follows calligraphic convention rather than spatial proximity, making purely geometric ordering cues ambiguous.

Multi-pass construction: After main ligature strokes are complete, writers return to add nuqta (character-distinguishing dots) and diacritical harakat. These secondary strokes are spatially interleaved with the primary glyph body but temporally subsequent, violating the spatial-contiguity assumption underlying our distance-based subsampling.

Right-to-left direction: The Pointer Network processes points without explicit writing-direction encoding. A left-to-right spatial bias can induce systematic errors, as rightmost points may be incorrectly predicted as stroke-initial even though Nastaliq proceeds right-to-left at the word level. The synthesis pipeline partially addresses this by sorting components rightmost-first and starting traversal from the top-right node, but this heuristic is not reflected in the Pointer Network's inductive bias.

B. Limitations of the Pointer Network

The pointer network implicitly assumes that temporal order is recoverable from spatial coordinates alone. For single-stroke patterns this assumption holds reasonably, but for Nastaliq's multi-pass, direction-mixed construction it is violated. The model achieves reasonable within-stroke ordering (reflected by improved adjacency coherence) but degrades at stroke boundaries where the pen lifts and re-approaches from a spatially distant position.

The $d_{\min} = 200$ subsampling may discard curvature information at tight Nastaliq turns, and centroid-normalisation discards absolute scale, removing glyph size as a stroke-identity cue.

C. Limitations of the VLM

Stroke coordinate sequences are fundamentally unlike natural language: they exhibit strong local spatial correlations, require precise numerical accuracy, and grow linearly in token count with image complexity. At a 40 ms sampling rate, a full line produces 500–2000 tokens of coordinate data, competing with the OCR transcription for generation budget within the model's context window.

D. Value of the Synthesis Pipeline

The Two-Thirds Power Law resampling produces velocity profiles consistent with the neuroscientific motor-control literature [10]. Qualitative visualisation of synthesised trajectories (colour-coded red at start to blue at end) shows smooth, continuous motion within ligature strokes and realistic inter-stroke pauses, confirming that the pipeline generates physically plausible temporal orderings from font glyphs. These synthesised trajectories can augment the training corpus for the Pointer Network, partially compensating for the limited size of TUKL OUHD-L.

X. Future Work

RTL-aware positional encoding: Both models should incorporate explicit right-to-left spatial priors. For the Pointer Network, a learned RTL coordinate embedding could encode column position in descending order. For the VLM, reversing the coordinate output order would align generation direction with Nastaliq writing convention.

Stroke boundary supervision: Augmenting the pointer loss with a stroke-boundary prediction head, predicting whether consecutive selected points belong to the same pen-down segment, would give the model a direct signal for pen-lift events rather than inferring them from spatial gaps.

Diacritic-aware two-pass decoding: A secondary lightweight decoder pass conditioned on first-pass base glyph trajectories should be explored for nuqta and harakat recovery, addressing the systematic diacritic omission observed in VLM outputs.

Coordinate compression: Replacing raw (x, y) integer pairs with delta-encoded relative displacements and spline fitting would substantially reduce trajectory token counts, improving VLM generation quality within context limits.

Dataset scale: The 2,403 training samples in TUKL OUHD-L are insufficient for robust generalisation. A data collection campaign targeting 10,000 or more line-level online samples across diverse writer demographics would benefit all components of VectUr.

XI. Conclusion

This paper presents **VectUr**, the first systematic investigation of online pen trajectory recovery for Urdu Nastaliq handwriting. VectUr comprises three components: a full Transformer encoder-decoder Pointer Network that learns to temporally order shuffled spatial pen-trace samples evaluated using Kendall's τ and adjacency coherence; a font-based synthesis pipeline using skeleton graph traversal and Two-Thirds Power Law resampling to generate training-grade trajectories from Nastaliq glyphs; and a QLoRA-fine-tuned Qwen3-VL-4B VLM trained on a three-stage curriculum for joint Urdu OCR and stroke coordinate generation.

The VLM achieves state-of-the-art Urdu OCR on the URTI benchmark (Books CER: 13.87 vs. 23.40 for Maqsood et al. [5]), validating the pre-training curriculum's effectiveness. The Pointer Network demonstrates the viability of permutation learning for stroke ordering, with adjacency coherence approaching 0.90 in overfit settings. Both trajectory recovery components fall short of deployment-quality outputs, with systematic failure modes tied to Nastaliq's multi-pass ligature construction, RTL writing direction, and limited training data. We release all code, dataset splits, and training logs as a reproducible open baseline.

TABLE VI
TUKL OUHD-L DATASET SPLIT STATISTICS (AFTER $d_{\min} = 200$ SUBSAMPLING)

Split	Samples	Avg. Seq. Length	Writers
Train	∼1,900	∼140 pts	Multiple
Validation	∼250	∼138 pts	Held-out
Test (leakproof)	∼253	∼135 pts	Writer-disjoint
Total	2,403	—	—

TABLE VII
TRAJECTORY EVALUATION METRICS IN VECTUR

Metric	Range	Interpretation
Kendall's τ	$[-1, 1]$	Global rank correlation; 1.0 = perfect order
Adj. Coherence	$[0, 1]$	Local stroke continuity; 1.0 = all neighbours preserved
Step-Offset Var.	$[0, N]$	Mean abs. position error vs. GT
OCR CER	$[0, \infty)$	Edit distance / GT length

APPENDIX

Given image dimensions $W \times H$, raw pixel coordinates (px, py) are normalised as:

$$x_{\text{norm}} = \left\lfloor \frac{px}{W} \times 1000 \right\rfloor, \quad y_{\text{norm}} = \left\lfloor \frac{py}{H} \times 1000 \right\rfloor \tag{8}$$

clipped to $[0, 1000]$. Each stroke segment is a space-separated sequence of `x,y` pairs; segments are separated by newline-prefixed `|` characters.

ACKNOWLEDGMENT

The authors thank the NUST School of Electrical Engineering and Computer Science for compute access and the participants who contributed handwriting samples to the TUKL OUHD-L dataset.

REFERENCES

[1] B. Mitrevski, A. Rak, J. Schnitzler, C. Li, A. Maksai, J. Berent, and C. Musat, "InkSight: Offline-to-online handwriting conversion by teaching vision-language models to read and write," *Transactions on Machine Learning Research*, 2025.

[2] Z. Jin, S. Desai, X. Chen, B. Fang, Z. Huang, Z. Li, and S. Liu, "TrInk: Ink generation with Transformer network," *arXiv:2508.21098*, 2025.

[3] S. Hanif and L. J. Latecki, "Strokes trajectory recovery for unconstrained handwritten documents with automatic evaluation," in *Proc. 12th Int. Conf. Pattern Recognition Applications and Methods (ICPRAM)*, pp. 661–671, SCITEPRESS, 2023. doi: 10.5220/0011700100003411

[4] B. Rabhi, A. Elbaati, H. Boubaker, U. Pal, and A. M. Alimi, "Multilingual handwriting recovery framework based on convolutional denoising autoencoder with attention model," *Multimedia Tools and Applications*, vol. 83, pp. 22295–22326, 2024. doi: 10.1007/s11042-023-16499-z

[5] M. Maqsood, S. Gillani, and S. Mehmood, "A unified deep learning architecture for Urdu OCR," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2023.

[6] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[7] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[9] Qwen Team, "Qwen3-VL Technical Report," Alibaba Group, 2025. [Online]. Available: https://qwenlm.github.io/blog/qwen3-vl/

[10] F. Lacquaniti, C. Terzuolo, and P. Viviani, "The law relating the kinematic and figural aspects of drawing movements," *Acta Psychologica*, vol. 54, no. 1–3, pp. 115–130, 1983.

[11] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1–2, pp. 81–93, 1938.

[12] A. Ul-Hasan, S. B. Ahmed, F. Shafait, M. Liwicki, and T. M. Breuel, "Efficient, high quality Urdu text recognition using scale- and rotation-invariant features," in *Proc. Int. Workshop on Document Analysis Systems (DAS)*, 2014.

[13] M. Z. Afzal, S. Capobianco, M. I. Malik, S. Marinai, T. M. Breuel, A. Dengel, and M. Liwicki, "Deep recurrent neural network-based features for Arabic handwriting recognition in different conditions," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2015.

[14] U. Pal, N. Tripathy, and B. B. Chaudhuri, "Handwritten Urdu script recognition: A survey," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007.

[15] H. Boubaker, A. Fadhel, A. El-Baati, H. El-Abed, and A. M. Alimi, "Arabic/Latin offline writer identification using fuzzy-logic," in *Proc. Int. Conf. Image Processing (ICIP)*, 2011.